Object Oriented Programming (CS1004)

Lecture 9
Arrays of objects and dynamically allocating objects

Lecture Contents

- Arrays of objects
- Dynamically allocating objects
- Dynamically allocating array of objects
- Destructors

Array of objects

- So far we declared only a single object of the class in our programs.
- What if we want multiple objects of the same class?
    - We can use array of objects.
- In array of objects each element in the array is an object of the same class.
- Efficient way to store and manage multiple objects of the same type.
- Helps organize data when dealing with multiple records (e.g., students, employees, products).
- In next slide we show how can we initialize, access and manipulate arrays of objects.

Arrays of Objects

```
class Point {
private:
   int x, y;  // Private data members
 public:
Point(int x = 0, int y = 0); // Constructor with default arguments
   int getX();  // Getter
   void setX(int x);  // Setter
   int getY();
   void setY(int y);
   void setXY(int x, int y);
   void print();
};
```

```
Point::Point(int _x, int _y){x = _x; y=_y;  }
int Point::getX() { return x; }
int Point::getY() { return y; }
void Point::setX(int x_) { x = x_; }
void Point::setY(int y_) { y = y_; }
void Point::setXY(int x_, int y_) { x = x_; y = y_; }
void Point::print() {
cout << "Point @ (" << x << "," << y << ")"; }
```

Arrays of Objects

- The below solution uses ***default constructor*** for all elements of the array
- This fails if there's no default constructor because C++ requires a constructor with no arguments when creating a static array.

```
int main() {
Point ptsArray[2];      // Array of Point objects
 ptsArray[0].print();    // Point @ (0,0)
 ptsArray[1].setXY(11, 11);
 ptsArray[1].print()     // Point @ (11,11)
}
```

initializing Arrays of Objects using non-default constructor

- The second method is to ***explicitly provide arguments*** for each object.
- In this case we have to provide parameterized constructor in the class.

```
int main() {
 Point ptsArray2[3] = { Point(21, 21), Point(22, 22), Point() };
 ptsArray2[0].print();   // Point @ (21,21)
 cout << endl;
 ptsArray2[0].print();   // Point @ (0,0)
```

Allocating object dynamically

- Similarly, we can allocate an object dynamically on the heap.

```
Int main() {
Point * ptrobj1 = new Point();//object created using default
constructor


Point *ptrobj2 = new Point(10,20); //object created using
parameterized constructor


//rest of the code


//make sure to deallocate memory


delete ptrobj1;
delete ptrobj2;
}
```

Arrays of dynamically allocated Objects

- We can dynamically allocate array of objects.
- In this way, the default constructor is called.

```
Point * ptrPtsArray3 = new Point[2];//objects created using default constructor
ptrPtsArray3[0].setXY(31, 31);
ptrPtsArray3[0].print();  // Point @ (31,31)

cout << endl;
ptrPtsArray3[1].setXY(32, 32);
ptrPtsArray3[1].print();            // Point @ (32,32)
cout << endl;

//do not forget to deallocate dynamically allocated memory.
delete[] ptrPtsArray3; // Free memory
```

Array of pointers of type rectangle

- Another way is to declare array of pointers of type rectangle.
- Then point each pointer to the dynamically allocated object.
- We can also use the parameterized constructor.

```
int main() {
Rectangle  *rec[3];
for(int i=0;i<3;i++)
  rec[i] = new Rectangle(2,5);//invoking constructr for each obj.
cout<<"Enough Rectangles, now going to out of scope: "<<endl
for(int i=0;i<3;i++)
  delete rec[i];//deallocating dynamically allocated objects
}
```

```
Rectangle width = 2 and lenth = 5
Rectangle width = 2 and lenth = 5
Rectangle width = 2 and lenth = 5
Enough Rectangles, going out of scope now:
```

Class exercise

- Create a class where students have attributes like name, rollNumber, and GPA. In main() allow dynamic allocation of an array of students based on user input (e.g., n students).

Destructors

- Destructor
  - is the member function of the class
  - used to destroy the object that has been created by a constructor
  - is called automatically when local objects go out of scope
  - perform termination housekeeping before the system reclaims the object's memory
  - Complement of the constructor
- Destructor naming:
  - Name is tilde (~) followed by the class name (i.e., ~Time())
    - Recall that the constructor's name is the class name
  - Receives *no parameters*, returns *no value*
  - One destructor per class
    - *No overloading* allowed

- If you do not explicitly provide a destructor, the compiler will create an empty destructor.

Syntax for user-defined Destructor

```
Classname::~dayofyear()  { }
```

- Cleanup is as important as initialization and is guaranteed through the use of destructors.
- Destructor never has any arguments, because it does not need any options.

Constructor/Destructor Example

```
class Employee {
    string name;
    int id;
    float salary
    public:
      Employee () {
      cout << "Employee's class object constructed"<<endl;
          }
      ~Employee () {
        cout << "Employee's class object destructed"<<endl;
          }
    };
    int main () {
        Employee emp;
    } // destructor will be called here
```

PrograProgram Output:

Employee's class object constructed

Employee's class object destructed

Simple Destructor: Another Example

```
class Rectangle {
public:
Rectangle(int w=5,int l=10){
Width=w; length=l; }
~Rectangle() {
cout<<"Rectangle object being destroyed: ";
private:
  int width;
  int length;
};
int main() {


Rectangle  r1;
Rectangle  r2(2,40);
Rectangle  r3(3,60);
//Destructors for objects are implicitly called here.
}
```

```
Rectangle width = 5 and lenth = 10
Rectangle width = 2 and lenth = 40
Rectangle width = 3 and lenth = 60
Rectangle is being destroyed:
Rectangle is being destroyed:
Rectangle is being destroyed:
```

Destructor Example: For array of objects

- The default constructor must be provided for creating array of objects:
- Destructor will be called automatically.
  - There is no need to call destructor explicitly.

```
int main() {
Employee Emp[3];//it will create three objects.
 Only default constructor will be called
Emp[0].setID(1);
Emp[1].setID(2);
Emp[2].setID(3);
} //destructors will be called implicitly
```

Object holding variable on the heap

- It is possible that when an object is created, one or more of its members are declared pointers.
- These pointers may point to dynamically allocated memory.

```
class dynamicvar {
private:
    int * ptr;
public:
    dynamicvar(int n) {
cout<<"in constructor: Allocating variable on the heap:
"<<endl;
    ptr = new int;
    *ptr = n; }
```

Object holding variable on the heap

- When an object goes out of scope, the dynamic storage that it "owns" must also be deallocated.
- this will not happen by default; we need to explicitly deallocate dynamic storage in the destructor.

```
class dynamicvar {
private:
    int * ptr;
public:
    dynamicvar(int n) {
    cout<<"in constructor: Allocating variable on the heap: "<<endl;
    ptr = new int;
    *ptr = n;  }
    ~dynamicvar(){
cout<<"In Destructor: deallocating variable:";
delete ptr;
};
int main() {
dynamicvar D1(5);
}//destructor is called automatically
```

Class exercise#2

- Write a class myarray. The constructor will dynamically allocate an array when an object of class myarray is created.
- Similarly, destructor will delete the dynamically allocated array when the object goes out of scope.

```
class myarray {
private:
    int * Arr;
    int size;
public:
    myarray(int); // constructor
    ~myarray(); // destructor
};
myarray::myarray(int n){
cout<<"In Constructor: Array is being allocated dynamically";
    Arr = new int[n];
    }
myarray::~myarray() {
cout<<"In Destructor: Dynamically allocated array is being deleted:";
delete[] Arr;
 }
int main() {
myarray A1(10);
}
```

Allocating entire object on the heap and deallocating the object

- Destructors for dynamically allocated object will be called when the object is deallocated.

```
Class DayofYear {
private:
  int day,month,year;
public:
DayofYear(int d,int m, int year) {
Day=d;month=m;year=y;}
~DayofYear(){
cout<<"DayofYear object being destroyed: "<<endl;
 }
};

main() {
  DayofYear *D1 = new DayofYear(10,3,2024);
} // what happens??
```

When is a destructor called for called for Dynamically allocated Object?

- Destructors shall not be explicitly called.
- The destructor DayofYear::~DayofYear() will automatically get called when you delete it using delete D1;
- **Remember: delete D1 does two things: it calls the destructor and it deallocates the memory.**

```
Class DayofYear {
private:
  int day,month,year;
public:
DayofYear(int d,int m, int year) {
Day=d;month=m;year=y;}
~DayofYear(){
cout<<"DayofYear object being destroyed: "<<endl;  }
};
main() {
   DayofYear *D1 = new DayofYear(10,3,2024);
          …
delete D1; //Automatically calls D1->~DayofYear()
}
```

Destructor Example: For dynamically allocated array of objects

- The default constructor must be provided for creating array of objects using below method:
- Use `delete` for deallocating the dynamic array of objects.
- Without using `delete`, destructors will not be called.

```
Employee *c = new Employee[3];//it will create three
 objects. Only default constructor will be called
Emp[0].setID(1);
Emp[1].setID(2);
Emp[2].setID(3);

delete [] c; //destructor will call here
```

Destructor Example: Array of pointers of type rectangle

```
class Rectangle {
public:
Rectangle(int w=5,int l=10){
Width=w;,length=l; }
~Rectangle() {
cout<<"Rectangle object being destroyed: ";
private:
  int width;
  int length;
};
int main() {
Rectangle  *rec[3];
for(int i=0;i<3;i++)
  rec[i] = new Rectangle(2,5);//invoking constructr for each obj.
cout<<"Enough Rectangles, now going to out of scope: "<<endl
for(int i=0;i<3;i++)
  delete rec[i];//destructors for the dynamically allocated objects
          called here automatically.
}
```

```
Rectangle width = 2 and lenth = 5
Rectangle width = 2 and lenth = 5
Rectangle width = 2 and lenth = 5
Enough Rectangles, going out of scope now:

Rectangle is being destroyed
Rectangle is being destroyed
Rectangle is being destroyed
```

Object holding variable on the heap

- When an object is deallocated, the dynamic storage that it "owns" must also be deallocated.
- This will not happen by default; we need to explicitly deallocate dynamic storage using delete or delete[].

```
class dynamicvar {
private:
    int * ptr;
public:
    dynamicvar(int) {
    cout<<"in constructor: Allocating variable on the heap: "<<endl;
    ptr = new int;
    *ptr = n;}
    ~dynamicvar(){
cout<<"In Destructor: Dynamically allocated variable is being deleted:";
delete ptr;
};

int main() {
dynamicvar *objptr = new dynamicvar(5);

delete objptr; //reclaiming the allocated memory
}
```